

SENSORES, ATUADORES E ARMAZENAMENTO

TEMA 1 – SENSORES E PROTOBOARD

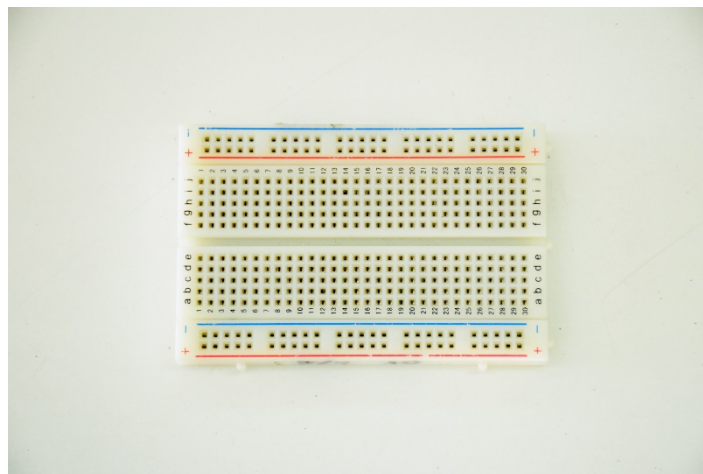
Um robô ou qualquer projeto com Arduino que necessite interagir com o meio ambiente precisa de sensores e atuadores. O Arduino interpreta os sinais provenientes dos sensores e os atuadores são responsáveis pelas “ações” que o projeto executará.

Segundo Stevan Junior (2015, p. 24), “sensor é um dispositivo capaz de sentir a variação de alguma grandeza física qualquer”.

Para essa disciplina, a metodologia a ser abordada compreende a explicação básica de atuação de um sensor, seu funcionamento e aplicações, e um exemplo prático. As principais informações que os sensores mais utilizados captam são: temperatura, umidade, luminosidade e distância.

Antes de iniciar o estudo dos sensores e fazer projetos, é preciso conhecer um dos componentes mais utilizados na eletrônica quando o assunto é montagem de circuitos eletrônicos: o protoboard.

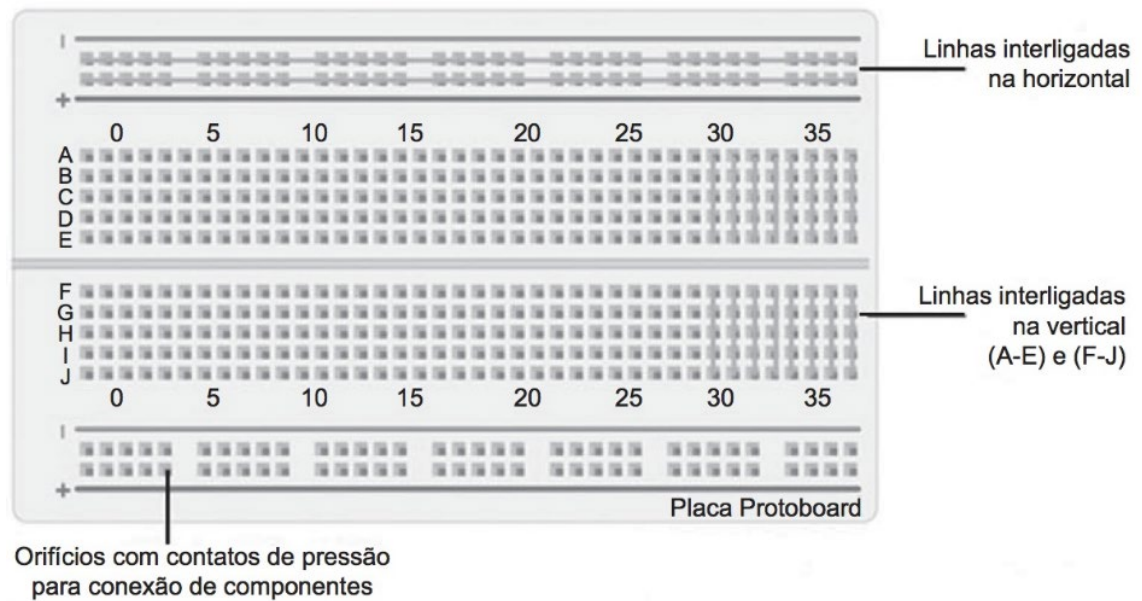
Figura 1 – Protoboard



Créditos: Wanasta Chaichaowarat/Shutterstock.

O protoboard também é conhecido como *matriz de contato*, *breadboard* ou *placa de ensaio*. Ele é uma placa com uma matriz de contatos que permite que os usuários montem e desmontem circuitos de forma fácil, pois não é necessário utilizar solda. Observe na Figura 2 como são interligadas as linhas do protoboard.

Figura 2 – Portos do protoboard



Fonte: Stevan Junior, 2015, p. 24.

Existem vários tamanhos de protoboards, mas todos apresentam o mesmo funcionamento. Nos protoboards menores, não há linhas interligadas horizontalmente. Cabe ao usuário definir qual o tamanho se aplica da melhor forma ao projeto, pois a funcionalidade será a mesma.

TEMA 2 – TIPOS DE SENSORES

Existem vários tipos de sensores a venda. Assim como o Arduino, a grande maioria dos sensores são em código aberto. Segundo Stevan Junior (2015, p. 25), pode-se classificar os tipos de sensores de diferentes formas:

- Quanto ao campo de atuação: sensores para biomedicina, meteorologia, consumo, automação.
- Quanto à aplicação, função que realizam ou que medem: sensores de pressão, aceleração, campo magnético, temperatura, capacidade térmica.
- Quanto ao princípio físico de funcionamento: sensores de transdução resistiva, transdução capacitiva, transdução indutiva, transdução piezoelétrica, transdução piezoresistiva, transdução fotovoltaica, transdução termoelétrica.
- Quanto à forma de energia de sinal que convertem: sensores de energia mecânica, magnética, radiante, térmica e elétrica (não há conversão da forma de energia do sinal).

Para se organizar, Stevan Junior (2015, p. 26) faz a seguinte subclassificação organizacional:

- Quanto à tecnologia: sensores indutivos, sensores capacitivos, sensores fotoelétricos, sensores resistivos e extensômetros, sensores piezoelétricos, sensores magnéticos e sensores ultrassônicos.
- Quanto à aplicação: sensores de posição/deslocamento, sensores de nível, sensores térmicos, sensores de umidade, sensores de pressão, sensores de gás, sensores de tensão elétrica, sensores de corrente elétrica, sensores de luz e sensores de som.

A seguir, vamos trabalhar com os tipos de sensores de cada subclassificação, e uma breve explicação. Iniciamos com a tecnologia:

- Sensores indutivos: percebem a alteração de um campo magnético.
- Sensores capacitivos: usados para armazenamento de energia entre duas placas paralelas.
- Sensores fotoelétricos: identificam a presença, a proximidade e/ou o deslocamento.
- Sensores resistivos e extensômetro: convertem um deslocamento físico linear ou angular em uma variação resistiva.
- Sensores piezoelétricos: têm a capacidade de converter energia mecânica (como pressão ou vibração) em energia elétrica.
- Sensores magnéticos: percebem um campo magnético.
- Sensores ultrassônicos: funcionam como detectores de objetos, permitindo medir a distância do objeto.

Quanto à aplicação:

- Sensores de posição / movimentação / deslocamento: identificam a posição, relativa ou absoluta, de um objeto.
- Sensores de nível: detectam o nível de fluidos ou tanques.
- Sensores térmicos: medem a temperatura.
- Sensores de umidade: medem a umidade.
- Sensores de pressão: medem a pressão, atmosférica ou de fluidos.
- Sensores de gás: existem vários sensores de gás, cada um para um tipo de gás.
- Sensores de tensão elétrica: medem a tensão elétrica.

- Sensores de luz: medem a intensidade luminosa.
- Acelerômetros e giroscópios: proporcionam um conhecimento completo de aceleração, velocidade, posição e orientação de um objeto.
- Sensores de som: captam sons de alta sensibilidade.

TEMA 3 – SENSOR DE LUMINOSIDADE

O sensor de luminosidade, representado na Figura 3, é conhecido pelo acrônimo LDR (Light Dependent Resistor), sendo um componente que varia em resistência conforme a luminosidade no ambiente em que se encontra. Quanto maior a quantidade de luz sobre ele, menor vai ser a resistência do LDR.

Figura 3 – Sensor LDR



Créditos: Cristian Storto/Shutterstock.

O sensor LDR é composto por uma área maior fabricado com um material semicondutor. Conforme a exposição do sensor à luz, a resistência elétrica vai variar, pois todo material semicondutor é sensível à radiação. Essa radiação não é necessariamente luminosa. A resistência elétrica do LDR sob luz intensa (10 Lux) pode variar entre 1.000 ohms e 10.000 ohms, e sem presença de luz é sempre maior que 200.000 ohms (Arduino, 2019).

Como o valor captado pelo sensor LDR apresenta variações dentro de um determinado intervalo (não apenas ligado ou desligado), sua conexão deve ser em uma porta analógica. A utilização mais comum de um sensor de luminosidade

é a detecção do amanhecer e do anoitecer. Quem nunca se perguntou como as luzes dos postes acendem e apagam sozinhas?

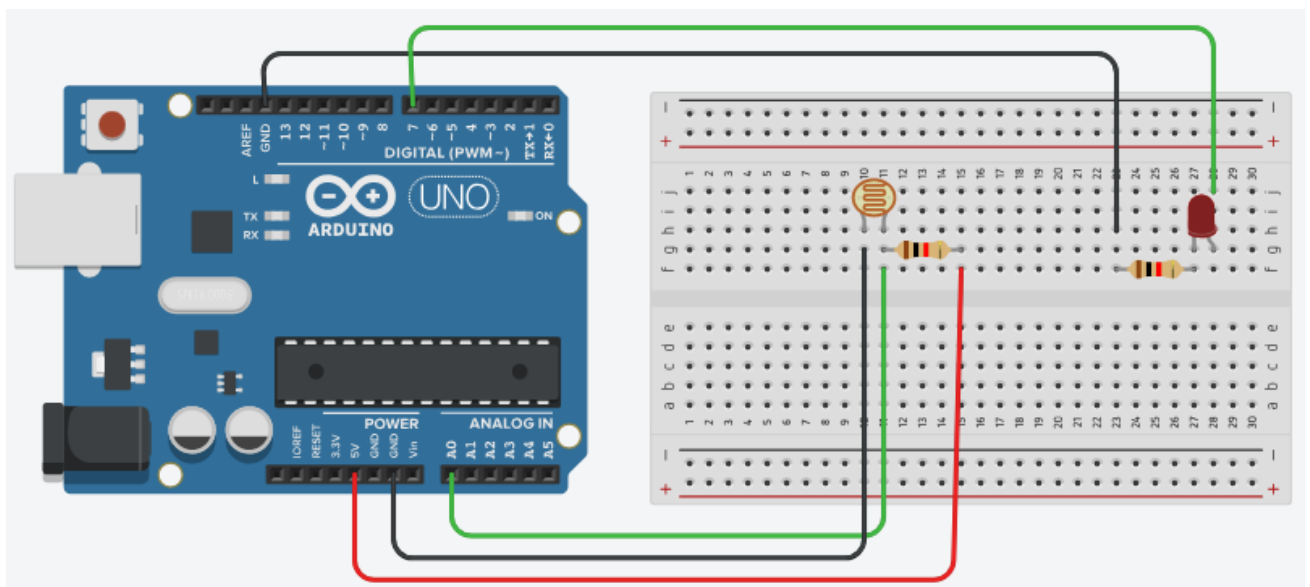
3.1 Projeto LDR

O projeto a ser desenvolvido com um sensor de luminosidade (LDR) consiste em ligar ou desligar um LED, dependendo da quantidade de luminosidade no ambiente. Sempre que for manusear o Arduino, é importante que ele não esteja ligado a nenhuma fonte de energia (USB ou Jack), assim você não terá problemas, como o risco de queimar o Arduino. Para a realização desse projeto, será necessário:

- Arduino
- Sensor LDR
- LED
- Protoboard
- Jumper's
- Resistor 300 Ohm
- Resistor 10k Ohm

O primeiro passo é fazer a montagem do hardware do projeto. A Figura 4 representa as ligações necessárias.

Figura 4 – Representação da montagem do Projeto LDR



Créditos: Lucas Rafael Filipak.

Repare, na Figura 4, que o LED é conectado a uma porta digital (pino 7) e o sensor LDR a uma porta analógica (pino A0). Preste bastante atenção às portas em que eles foram conectados, pois elas vão ser programadas no software. Após a montagem do hardware, é preciso programar (criar o software).

Figura 5 - Representação do sketch do Projeto LDR

```
1 //Projeto Sensor LDR
2
3 int led = 7; //Led conectado no pino 7
4 int ldr = 0; //LDR conectado no pino analógico 0
5 int valorLdr = 0; //Inicializar a variável
6
7 void setup() {
8     pinMode(led,OUTPUT); //configurando o pino 7 como saída
9     Serial.begin(9600); //Inicia a comunicação serial
10 }
11
12 void loop() {
13     valorLdr = analogRead(ldr); //leitura do valor do sensor LDR
14
15     //se o valor lido for maior que 500, liga o led
16     if (valorLdr >= 500) digitalWrite(led,HIGH);
17     // senão, apaga o led
18     else digitalWrite(led,LOW);
19
20     delay(100);
21 }
```

Crédito: Lucas Rafael Filipak.

A variável “led”, declarada na linha 3, está recebendo o valor 7. Isso significa que ela vai se comunicar com algo ligado ao pino 7 do Arduino. A variável “ldr” está recebendo o valor 0, ou seja, está conectada ao pino 0. Foi criada a variável “valorLdr” para receber o valor lido pelo sensor. Essa variável foi inicializada com o valor 0. Nesse momento, quando apenas são declaradas as variáveis, não é possível saber se os pinos informados são digitais ou analógicos, ou ainda se não é apenas um valor inicial de uma variável (como exemplo da variável valorLdr).

A função setup() foi explicada anteriormente. Ela serve para definir as configurações iniciais. Na linha 8 da Figura 5, a variável “led” foi definida como OUTPUT, pois terá apenas a função de mostrar um resultado e não de fazer a

captação de algum valor. O comando da linha 9 informa (inicia) ao hardware que haverá uma comunicação com a porta serial.

Dentro da função `loop()`, em que o programa ocorre de fato. Na linha 13, a variável “valorLdr” recebe o valor que foi lido, analogicamente¹, no sensor que está plugado na porta analógica 0 (que foi definida na linha 4). Somente na linha 13 foi possível saber que a variável “valorLdr” receberia um valor proveniente de uma entrada analógica. Já sabemos qual é o valor lido pelo sensor (linha 13). Na linha 16, existe um comando decisão, que traduzindo seria: se o valor que foi captado e está armazenado na variável “valorLdr” for maior ou igual à 500, então ligue a variável “led” (foi definido, na linha 3, que a variável “led” é o sensor que está plugado no pino digital 7), senão (linha 18), desligue a variável “led”.

Note que apenas nas linhas 16 e 18 é possível saber que a variável “led”, que está ligada na porta 7, é do tipo digital, pois é utilizada a função `digitalRead()`.

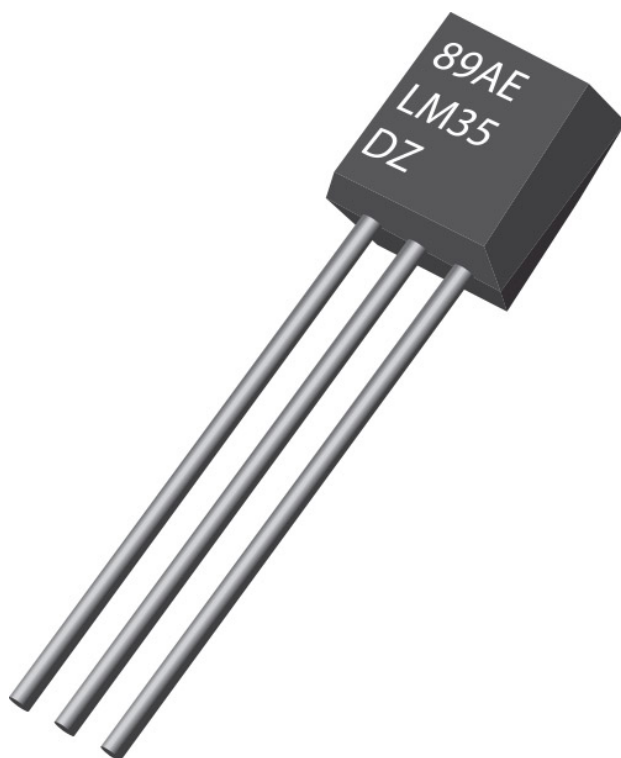
Assim como o sensor de luminosidade foi utilizado para ligar ou desligar um led (ou uma lâmpada), ele pode também ligar ou desligar qualquer equipamento que estiver plugado em uma porta digital. Afinal, no programa que foi utilizado como exemplo (Figura 5), nota-se que não há nenhum comando específico para ligar o led, mas sim um comando que liga o que estiver plugado na porta digital especificada (porta 7, no exemplo).

TEMA 4 – SENSOR DE TEMPERATURA (LM35)

Seguindo a mesma linha das portas analógicas, vamos apresentar um dos sensores que são utilizados para fazer a medição da temperatura de um ambiente. Digo *um* dos sensores, pois o LM35 é o mais utilizado e o mais barato sensor para essa função. Allan Mota (2017) diz que “O sensor LM35 é um sensor de precisão que apresenta uma saída de tensão linear proporcional à temperatura em que ele se encontrar no momento, tendo em sua saída um sinal de 10mV para cada Grau Célsius de temperatura.”

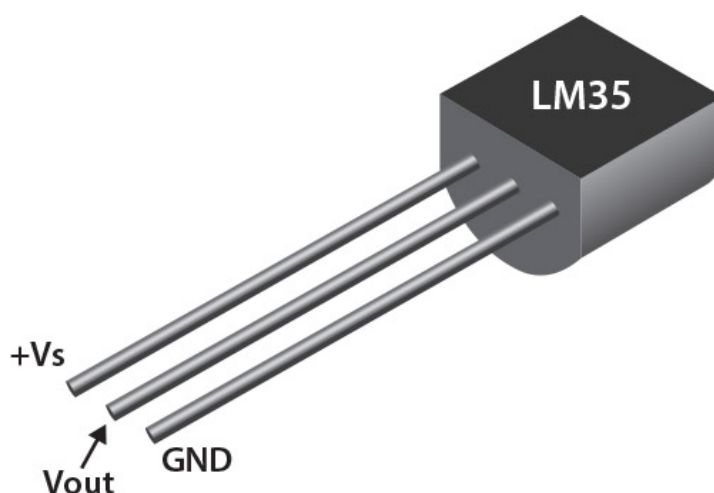
¹ `AnalogRead()` é a função que faz a leitura de valor de um sensor que está plugado em uma porta analógica.

Figura 6 – Sensor de temperatura LM35



Repare, na Figura 6, que o sensor possui 3 pinos (pernas). O primeiro pino é a alimentação, o segundo pino é o que vai transmitir as informações, e o terceiro pino é o terra (GND). Repare essa configuração na Figura 7:

Figura 7 - Função de cada pino do sensor LM35



É importante saber qual a função de cada pino/entrada de um sensor, pois com essa informação é feita a ligação na placa do Arduino. Por exemplo, se você ligar o pino 2 (que transmite os dados) na alimentação, o sensor não vai funcionar, podendo até queimar.

O sensor LM35 não capta a temperatura diretamente em Celsius, ele capta um valor que varia entre 0 e 1023, em que: 0 corresponde a 0V; 1023 corresponde a 5V.

O Arduino divide 5Volts, que é o maior valor que ele consegue ler, em 1023 partes iguais, e informa quantas partes tem o valor que ele está medindo. Para ficar mais claro, observe atentamente o projeto.

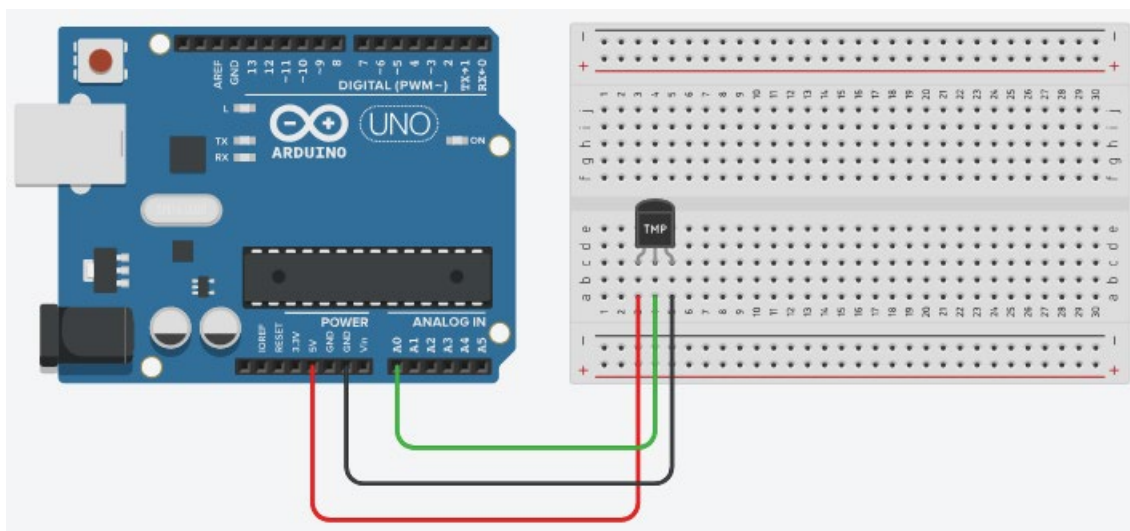
4.1 Projeto LM35

O projeto consiste em montar um protótipo que vai efetuar, em um intervalo de 3 segundos, a leitura da temperatura ambiente. O resultado será mostrado no “monitor serial”. Para a realização desse projeto, será necessário:

- Arduino
- Sensor LM35
- Protoboard
- Jumper's

O primeiro passo é fazer a montagem do hardware do projeto. A Figura 8 representa as ligações necessárias para o projeto.

Figura 8 – Representação da montagem do Projeto LM35



Créditos: Lucas Rafael Filipak.

Repare na ligação dos pinos – você pode comparar com a Figura 7. O primeiro pino está conectado com o jumper vermelho na alimentação, o segundo pino está conectado com o jumper verde na porta analógica (A0), e o terceiro pino

está conectado com o jumper preto no GND. Após a montagem do hardware, é preciso programar o sketch (criar o software).

Figura 9 – Representação do sketch do Projeto LM35

```
1 //Projeto Sensor LM35
2
3 const int LM35 = A0; // Define que o sensor estará no pino analógico A0
4 float temperatura; // Variável que armazenará a temperatura medida
5
6 void setup() {
7   Serial.begin(9600); // Inicia a comunicação serial
8 }
9
10 void loop() {
11   // Fórmula para converter para celsius
12   temperatura = (float(analogRead(LM35))*5/(1023))/0.01;
13   Serial.print("Temperatura: ");
14   Serial.println(temperatura); //Imprime no monitor a temperatura lida
15   delay(3000);
16 }
```

Crédito: Lucas Rafael Filipak.

Na linha 3 da Figura 9, a constante LM35 foi definida como A0, o que quer dizer que ela vai estar conectada ao pino analógico A0. Essa constante será responsável por fazer a medição do sensor. Na linha 4, foi criada a variável “temperatura”, que vai receber (na linha 12) o valor da temperatura convertida para celsius. Como já vimos no projeto anterior, a linha 7 inicia a comunicação serial. Na linha 12, está a fórmula utilizada para fazer a conversão do valor lido pelo sensor (que varia entre 0 e 1023) para celsius. Perceba a origem da fórmula:

$$\text{Tensão A0} = (\text{Valor captado na A0}) * (5 / 1023)$$

O cálculo da tensão utiliza 5 / 1023. O 5 representa o valor máximo em Volts que o Arduino consegue manipular, e 1023 o valor máximo que o sensor consegue captar.

$$\text{Temperatura} = \text{Tensão A0} / 10\text{mV}$$

Sabe-se que cada 1° C corresponde a 10mV. Então, o cálculo da temperatura é o valor da tensão dividido por 10mV. Logo, pode-se juntar as duas informações em uma mesma fórmula:

$$\text{Temperatura} = [(\text{Valor captado na A0}) * (5 / 1023)] / 10\text{mV}$$

Repare na Figura 10, que representa a linha 12 do sketch, que a fórmula foi passada para a linguagem de programação.

Figura 10 – Representação da fórmula na programação do Arduino

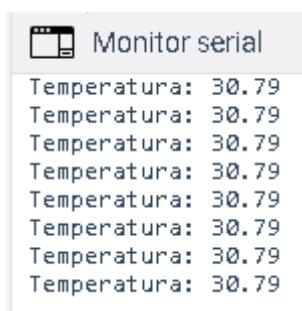
```
11 temperatura = (float(analogRead(LM35))*5/(1023))/0.01;
```

Crédito: Lucas Rafael Filipak.

O valor que o Arduino retorna é do tipo inteiro, sendo necessário transformar o valor inteiro em um valor decimal (racional). Segundo Mota (2017), em programação, “quando multiplicamos uma variável inteira por uma variável racional, o programa considera que o resultado deve ser inteiro, eliminando a parte decimal da variável. Dessa forma, para que tenhamos um resultado racional, devemos transformar o número inteiro em um número racional.” Para fazer essa transformação, colocamos a função `analogRead()` dentro da função `float()`.

Nas linhas 13 e 14 da Figura 9, o comando `Serial.print` vai “imprimir”/mostrar o resultado no Monitor Serial. A leitura é feita a cada 3 segundos, conforme definido na linha 15. Observe a Figura 11, que representa como é mostrado o resultado no Monitor serial.

Figura 11 – Resultado da execução do Projeto LM35



Monitor serial

```
Temperatura: 30.79  
Temperatura: 30.79  
Temperatura: 30.79  
Temperatura: 30.79  
Temperatura: 30.79  
Temperatura: 30.79  
Temperatura: 30.79  
Temperatura: 30.79
```

Crédito: Lucas Rafael Filipak.

A Figura 11 é a representação do Monitor Serial do Tinkercad2. O Tinkercad é um simulador on-line e está sempre atualizado, permitindo que o usuário faça a programação tradicional (essa mostrada nesse curso) que utiliza

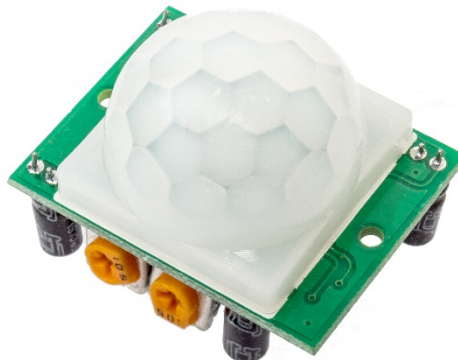
² <https://www.tinkercad.com> é um simulador on-line e gratuito com base no qual é possível montar o hardware, programar o software e verificar o funcionamento do protótipo.

linguagem de programação, mas também permite que o usuário faça a programação por blocos (Scratch).

TEMA 5 – SENSOR DE PRESENÇA – PIR HC-SR501

Uma área que está sendo muito explorada pelos amantes da tecnologia e/ou da robótica é a automação residencial. Um dos sensores mais utilizados nessa área é o sensor de presença, que é também conhecido como *sensor de movimento*. Como o próprio nome sugere, esse tipo de sensor detecta se há presença ou movimento dentro do raio de atuação do sensor. Observe a Figura 12.

Figura 12 – Representação do sensor PIR HC-SR501



Créditos: Pozdeyev Vitaly/Shutterstock.

Os sensores de presença do tipo PIR apresentam um sensor piroelétrico, que é capaz de detectar diferentes níveis de irradiação infravermelha emitidas pelo corpo humano. Segundo Madeira (2017):

Quando alguém passa na frente desse sensor, primeiramente a pessoa passará na zona de detecção do primeiro elemento piroelétrico e depois na zona de detecção do segundo elemento. Quando a pessoa passa pelo primeiro, ele gera um pulso de tensão na saída, quando ele passa na frente do outro, ele gera um pulso de tensão de sinal contrário.

5.1 Projeto PIR HC-SR501

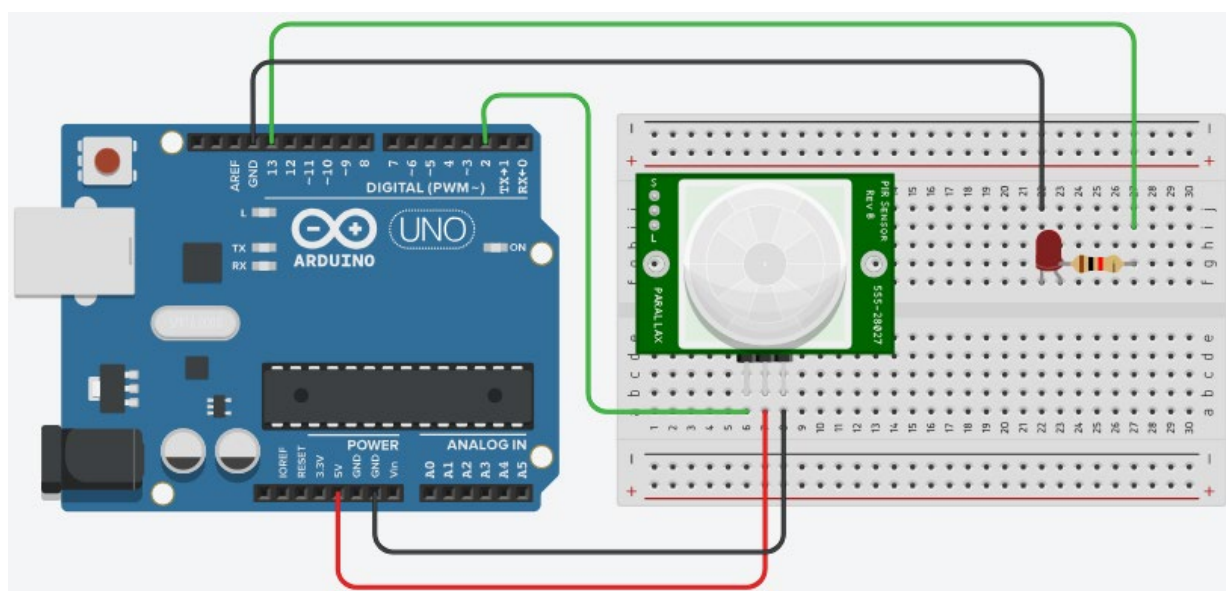
Os sensores de presença podem ser usados em muitos projetos diferentes, mas as duas principais utilizações desses sensores são: controlar um sistema de iluminação e interagir com um sistema de alarme. Nesse projeto, será feito o

protótipo de um sistema de iluminação. Para a realização desse projeto, será necessário:

- Arduino
- Sensor PIR HC-SR501
- Protoboard
- Jumper's
- LED
- Resistor 3300hm

O primeiro passo é fazer a montagem do hardware do projeto. A Figura 13 representa as ligações necessárias para o projeto.

Figura 13 – Representação da montagem do Projeto PIR HC-SR501



Crédito: Lucas Rafael Filipak.

Analisando o hardware, repara-se que os dois sensores utilizam as portas digitais e estão conectados com jumpers da cor verde. O LED precisa de um resistor, pois o seu valor de alimentação é muito pequeno, e a ausência de um resistor pode queimá-lo ou no mínimo reduzir a sua vida útil. O protótipo da Figura 13 foi desenvolvido no Tinkercad, e cada sensor apresenta ligações específicas. Segundo o Tinkercad, o sensor de presença tem três pinos. O primeiro é utilizado para a transmissão de dados, e vai conectado ao pino digital, o pino do meio é a alimentação e foi conectado ao Arduino por meio do jumper vermelho, e o terceiro

pino é o GND que foi conectado com o jumper preto. Após a montagem do hardware, é preciso programar o sketch.

Figura 14 – Representação do sketch do Projeto PIR HC-SR501

```
1 //Projeto sensor PIR HC-RS501
2
3 int val = 0;
4
5 void setup() {
6     pinMode(13, OUTPUT);
7     pinMode(2, INPUT);
8 }
9
10 void loop(){
11     val = digitalRead(2);
12     if (val == HIGH) {
13         digitalWrite(13, HIGH);
14         delay(1000);
15     }else {
16         digitalWrite(13, LOW);
17         delay(1000);
18     }
19 }
```

Crédito: Lucas Rafael Filipak.

No início do sketch da Figura 14, foi declarada a variável “val”, e inicializada com o valor 0 (zero). Essa variável vai receber na linha 11 a leitura digital informada pelo sensor, que está conectado na porta digital 2. O LED foi conectado ao pino 13 e configurado como saída (OUTPUT); o sensor foi conectado ao pino 2 e foi configurado como entrada. Dentro do loop, a leitura digital é feita e armazenada na variável “val”. Após armazenamento, a variável “val” é testada para saber qual é o seu conteúdo. Ela pode possuir apenas dois conteúdos: HIGH ou LOW. Se o conteúdo dela for HIGH, que no caso significa algum movimento captado pelo sensor, o LED vai acender e desligar após 1 segundo. Se o conteúdo da variável testada (val) for LOW, o LED permanecerá apagado. Esse teste de movimento é repetido de 1 em 1 segundo.

REFERÊNCIAS

ARDUINO. **Photoconductive cell**. Disponível em: <<https://www.arduino.cc/documents/datasheets/LDR-VT90N2.pdf>>. Acesso em: 15 maio 2019.

MADEIRA, D. Sensor de presença com Arduino | PIR HC-SR501. **Portal Vida de Silício**, 2017. Disponível em: <<https://portal.vidadesilicio.com.br/sensor-de-presenca-hc-sr501/>>. Acesso em: 15 maio 2019.

MOTA, A. Sensor de temperatura LM35: Medindo temperatura com Arduino. **Vida de Silício**, 2017. Disponível em: <<https://portal.vidadesilicio.com.br/lm35-medindo-temperatura-com-arduino/>>. Acesso em: 15 maio 2019.

STEVAN JUNIOR, S. L. **Automação e instrumentação industrial com Arduino**: teoria e projetos. São Paulo: Érica, 2015.